INFRASTRUCTURE RECREATION GUIDE

On-Call Assistant

Slack-native retrieval-augmented incident assistant on AWS

Document version 1.0 · 8 September 2026

Target audience: client platform / DevOps engineering

Scope: clean-room deployment into a customer AWS account and Slack workspace

Derived from the deployed implementation (`src/oncall/lambdas/`)

Contents

1. Executive summary

An on-call team's institutional knowledge — the root causes and fixes for recurring infrastructure and application incidents — typically exists only as unstructured Slack history. It is effectively unsearchable under incident pressure, so the same problems are re-diagnosed from scratch by whoever happens to be on call.

The On-Call Assistant converts that history into a curated, searchable knowledge base and serves it back inside Slack. Every message in a monitored channel is accumulated into a per-thread document. When a thread reaches a resolution, a large language model distils it into a structured case — issue, affected service, root cause, troubleshooting steps, solution — with secrets and personal data redacted and a confidence score attached. Only resolved, sufficiently confident cases enter the knowledge base. An engineer can then mention the bot in Slack and receive a grounded answer drawn exclusively from those past cases, each claim carrying a clickable link to the original thread so it can be verified.

The system is fully serverless: two AWS Lambda functions, an S3 bucket, and an Amazon Bedrock Knowledge Base backed by S3 Vectors. There are no servers, no vector database to operate, and no idle cost floor. This document contains everything required to recreate it in a customer environment.

How to read this document. Values specific to an environment appear as highlighted placeholders such as `<S3_BUCKET>`. Appendix A lists every placeholder and what to substitute. Items that could not be determined from the source repository are marked **TO CONFIRM** and collected in Appendix E.

2. Architecture overview

Two pipelines share one knowledge base. The **write path** turns Slack conversation into curated cases; the **read path** answers questions from those cases. Both run entirely on managed services.

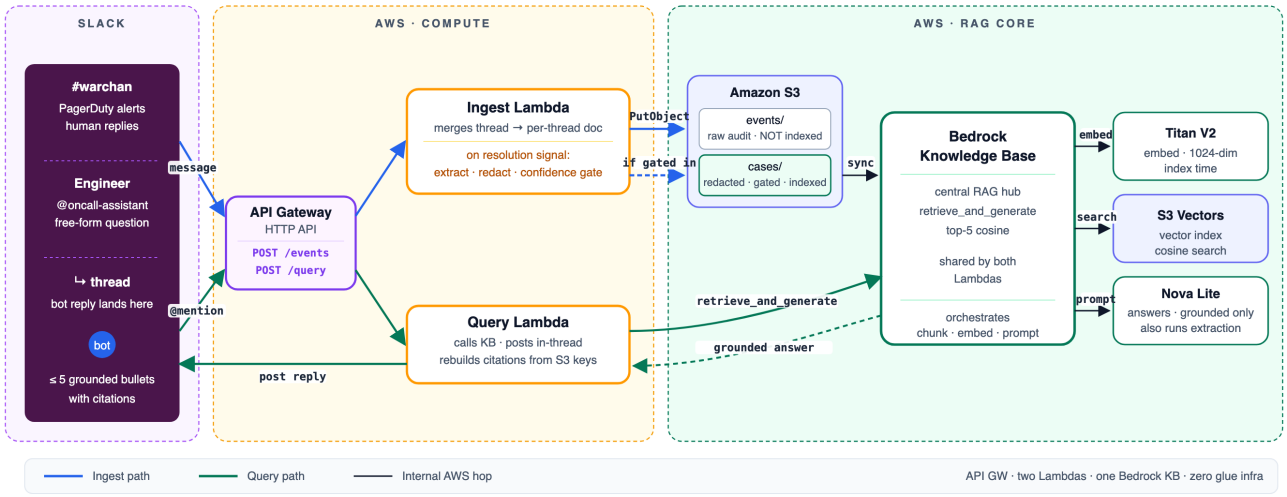

Figure 1 — End-to-end architecture. Slack events reach two Lambda functions; S3 holds two prefixes with different trust levels; the Bedrock Knowledge Base indexes only the curated one.

2.1 The two S3 prefixes — the most important design decision

> **Critical.** The bucket holds two prefixes that must never be confused. `events/` contains the *raw* Slack thread documents exactly as posted, including any credentials, tokens, customer identifiers or personal data an engineer pasted into the channel. It is an audit trail and **must never be configured as a Knowledge Base data source**. `cases/` contains only model-extracted, redacted, confidence-gated case records, and is the sole data source for the Knowledge Base. Pointing the data source at `events/` would make every unredacted secret in the channel's history retrievable through the bot.

2.2 Write path — ingestion and extraction

Every `message.channels` event is appended to its thread's JSON document under `events/{channel_id}/{thread_ts}.json`, keyed on the thread timestamp so replies land in their parent's file. Each message is classified by keyword into a role — `alert`, `investigation`, `action`, or `resolution`. Root messages are always `alert`.

Extraction is *conditional*: only when a threaded reply is classified `resolution` is the entire thread sent to Bedrock for distillation. The resulting case is written to `cases/` only if it passes the quality gate, and only then is a Knowledge Base ingestion job started. Ordinary chatter therefore costs one S3 write and nothing else.

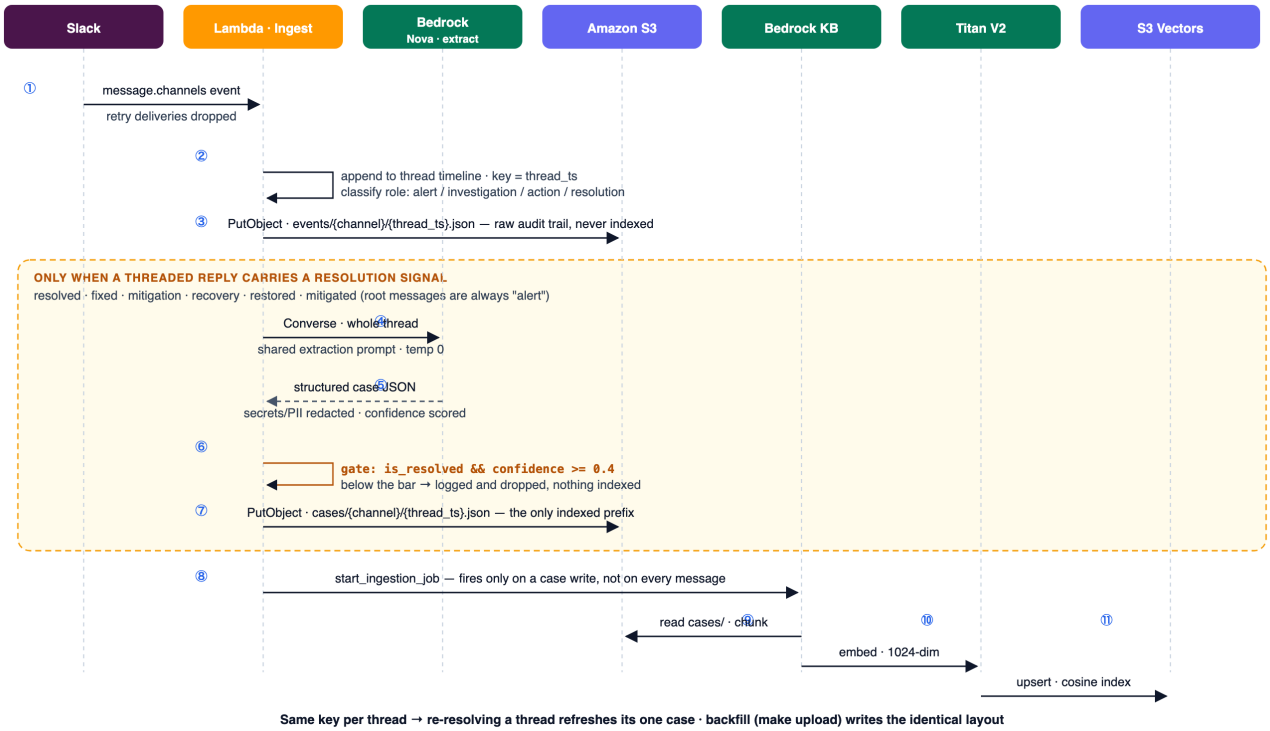

Figure 2 — Write path. The amber band contains the work that runs only on a resolution signal: extraction, redaction, the confidence gate, the curated write, and the knowledge-base sync.

Trigger word (in a threaded reply)	Effect
`resolved`, `fixed`, `mitigation`, `recovery`, `restored`, `mitigated`	Message classified `resolution` → the whole thread is extracted, gated, and (if it passes) indexed.

Matching is case-insensitive substring matching on the message text. Because the case is keyed on the thread timestamp, re-resolving a thread overwrites its single case rather than creating a duplicate — posting another resolution reply is also the supported way to re-extract a thread after improving the prompt.

2.3 Read path — answering questions

When an engineer mentions the bot, the query Lambda verifies the Slack signature, strips the mention, and — if the question was asked inside a thread — fetches the parent message to use as incident context. It then calls Bedrock `retrieve_and_generate`, which performs the vector search and the generation in a single call, and posts the answer as a threaded reply.

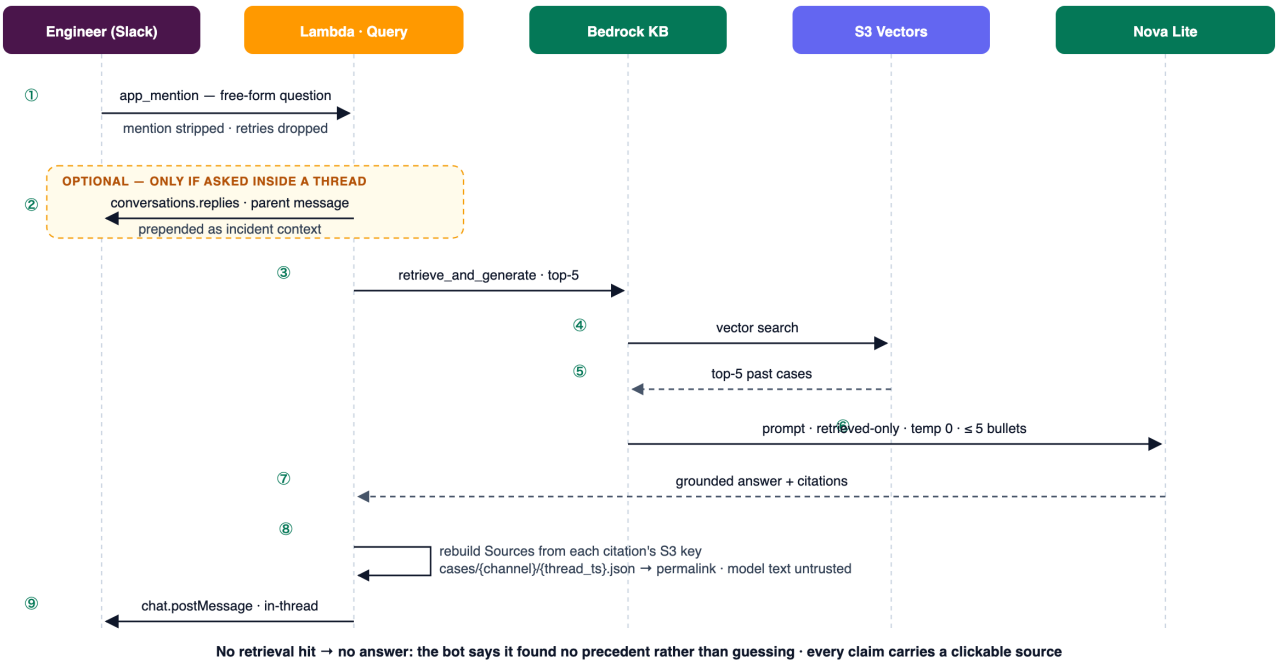

Figure 3 — Read path. Citations are reconstructed in application code rather than trusted from the model's output.

Citations are rebuilt, not trusted. In production the model proved unreliable at transcribing source links: it sometimes emitted internal citation markers such as `%[2]%`, and the retrieved chunks were sometimes split before the record's permalink field or returned with no chunk text at all. The query Lambda therefore discards whatever source list the model wrote and reconstructs it from the deduplicated union of (1) permalinks derived from each citation's S3 object key, (2) permalink text found in chunk contents, and (3) any URLs the model transcribed. Key-derived links are the reliable path, which is why `SLACK_WORKSPACE_URL` must be configured on the query Lambda.

2.4 Component view

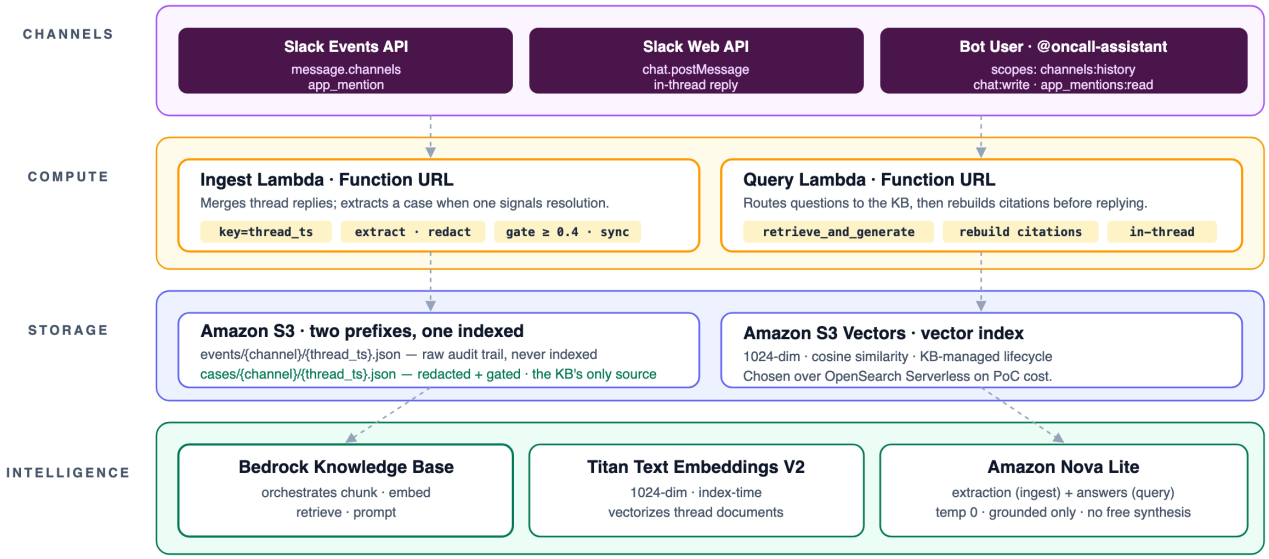

Figure 4 — Components by layer, with each one's responsibilities.

3. Prerequisites

Requirement	Detail
AWS account	With permission to create S3 buckets, Lambda functions, IAM roles, and Amazon Bedrock Knowledge Bases.
AWS region	A single region, `<AWS_REGION>`, offering Bedrock Knowledge Bases *and* S3 Vectors. Confirm availability before committing; the reference deployment runs in `us-east-2`.
Bedrock model access	Two enabled models: a **text-generation** model reachable through the Converse API (used for both extraction and answering), and **Titan Text Embeddings V2** (`amazon.titan-embed-text-v2:0`) for indexing. Request access in the Bedrock console before deploying — approval is not always instant.
Slack workspace	Admin rights to create and install an app, plus the ID of the channel to monitor (`<CHANNEL_ID>`, visible under channel → View details).
Local tooling	Python 3.10+, `make`, and `zip` to build deployment packages. AWS CLI for verification.
Source repository	The application code. Deployment packages are produced with `make lambda_zips`.

Choose the generation model deliberately. `BEDROCK_MODEL_ID` must be a Converse-capable *generation* model. Supplying an embeddings model ID here is a common and confusing failure: embeddings models cannot produce text, so extraction fails on every resolved thread while ingestion continues to look healthy. If the model is exposed as a cross-region *inference profile*, note both its profile ID and the underlying foundation-model ID — IAM needs both (see §5).

4. AWS resource inventory

Resource	Purpose	Settings that matter
S3 bucket `<S3_BUCKET>`	Holds raw thread documents and curated cases.	Two prefixes: `events/` (raw audit, never indexed) and `cases/` (curated, indexed). Block all public access. Enable default encryption (SSE-S3 minimum). Versioning recommended so a bad extraction can be rolled back.
Bedrock Knowledge Base	Chunking, embedding, retrieval, and answer generation.	Embeddings model: Titan Text Embeddings V2. Vector store: **S3 Vectors**. Records its own service role needing read access to the bucket.
KB data source	Tells the knowledge base what to index.	**S3 URI must be** `s3://<S3_BUCKET>/cases/` — never the bucket root and never `events/`. Default chunking and parsing are sufficient.
Ingestion Lambda `<EVENTS_FN_NAME>`	Receives Slack message events, maintains thread documents, runs extraction, writes cases, triggers sync.	Python 3.12+. Handler `lambda_function.lambda_handler`. **Timeout ≥ 60 s.** 256 MB memory recommended. Invoked over HTTPS (see §6).
Query Lambda `<QUESTIONS_FN_NAME>`	Receives bot mentions, queries the knowledge base, posts answers.	Python 3.12+. Handler `lambda_function.lambda_handler`. **Timeout ≥ 60 s.** 256 MB memory recommended.
IAM roles (×2)	Execution roles, one per Lambda.	Least-privilege policies in §5. Do not share one role between the two functions.
HTTPS endpoints	Public entry points Slack posts events to.	Lambda Function URLs or an API Gateway HTTP API — see §6 and Appendix E.
CloudWatch log groups	Operational visibility; the primary diagnostic surface.	`/aws/lambda/<function-name>`, created automatically. Set a retention policy (30–90 days) — the default is "never expire".

> **The 60-second timeout is not optional.** Both functions make Bedrock calls that routinely take 2–20 seconds. AWS Lambda's default timeout is 3 seconds, which kills the invocation mid-call. This failure is silent: no exception is logged, only `Status: timeout` in the final `REPORT` line. The observed symptom is "the bot never replies" or "cases never appear" with apparently clean logs.

5. IAM policies

Paste these as inline policies on each function's execution role, substituting the placeholders. Both also need the basic Lambda logging permissions shown.

5.1 Ingestion Lambda role

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Logs",
      "Effect": "Allow",
      "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "arn:aws:logs:<AWS_REGION>:<AWS_ACCOUNT_ID>:*"
    },
    {
      "Sid": "ReadWriteThreadDocsAndCases",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": [
        "arn:aws:s3:::<S3_BUCKET>/events/*",
        "arn:aws:s3:::<S3_BUCKET>/cases/*"
      ]
    },
    {
      "Sid": "ListBucketSoMissingKeysReturnNoSuchKey",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::<S3_BUCKET>"
    },
    {
      "Sid": "TriggerKnowledgeBaseSync",
      "Effect": "Allow",
      "Action": [
        "bedrock:StartIngestionJob",
        "bedrock:GetIngestionJob",
        "bedrock:ListIngestionJobs"
      ],
      "Resource": "arn:aws:bedrock:<AWS_REGION>:<AWS_ACCOUNT_ID>:knowledge-base/<KB_ID>"
    },
    {
      "Sid": "InvokeExtractionModel",
      "Effect": "Allow",
      "Action": "bedrock:InvokeModel",
      "Resource": [
        "arn:aws:bedrock:<AWS_REGION>:<AWS_ACCOUNT_ID>:inference-profile/
<INFERENCE_PROFILE_ID>",
        "arn:aws:bedrock:*::foundation-model/<FOUNDATION_MODEL_ID>"
      ]
    }
  ]
}
```

Two subtleties in this policy, both discovered in production.

- **Both Bedrock ARNs are required** when using a cross-region inference profile. The profile routes the request to the underlying foundation model, potentially in another region, so permission on the profile alone yields `AccessDeniedException` on `InvokeModel`. The wildcard region in the foundation-model ARN is deliberate.
- `s3:ListBucket` **matters even though the code never lists a bucket.** Without it, S3 reports a missing object as `AccessDenied` rather than `NoSuchKey`. The handler treats a genuinely absent thread document as "new thread", so a permissions gap would be silently misread as a fresh conversation, resetting timelines.

5.2 Query Lambda role

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Logs",
      "Effect": "Allow",
      "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "arn:aws:logs:<AWS_REGION>:<AWS_ACCOUNT_ID>:*"
    },
    {
      "Sid": "QueryKnowledgeBase",
      "Effect": "Allow",
      "Action": ["bedrock:RetrieveAndGenerate", "bedrock:Retrieve"],
      "Resource": "arn:aws:bedrock:<AWS_REGION>:<AWS_ACCOUNT_ID>:knowledge-base/<KB_ID>"
    },
    {
      "Sid": "InvokeAnswerModel",
      "Effect": "Allow",
      "Action": "bedrock:InvokeModel",
      "Resource": [
        "arn:aws:bedrock:<AWS_REGION>:<AWS_ACCOUNT_ID>:inference-profile/
<INFERENCE_PROFILE_ID>",
        "arn:aws:bedrock:*::foundation-model/<FOUNDATION_MODEL_ID>"
      ]
    }
  ]
}
```

`bedrock:Retrieve` is separate from `bedrock:RetrieveAndGenerate` and is required: when an answer comes back with zero citations the Lambda issues a diagnostic retrieval to distinguish "the knowledge base returned nothing" from "it returned chunks the model declined to use". Omitting it degrades troubleshooting but does not break answering.

The Knowledge Base's own service role — created with the knowledge base — separately needs read access to `s3://<S3_BUCKET>/cases/` and permission to invoke the embeddings model. The console wizard configures this automatically; verify it if you provision by IaC.

6. Slack application setup

Slack must deliver two different event types to two different HTTPS endpoints: `message.channels` to the ingestion Lambda and `app_mention` to the query Lambda.

> **TO CONFIRM — event delivery topology.** A single Slack app supports exactly *one* Event Subscriptions Request URL, but this system has two independent endpoints. The reference deployment's exact wiring is not captured in the source repository. Two workable topologies:
>
> - **Two Slack apps (recommended).** App 1 "ingestion" subscribes to `message.channels` and points at the ingestion endpoint. App 2 "assistant" subscribes to `app_mention`, points at the query endpoint, and owns the bot user engineers mention. Each app has its own signing secret — set the correct one on the corresponding Lambda.
> - **One app plus a dispatcher.** A single Request URL fronted by API Gateway invoking a small dispatcher that fans the event out to both functions. Fewer Slack apps, one more moving part.
>
> Confirm which topology the reference deployment uses and delete the other before issuing this document to the client. Both handlers defensively ignore event types that are not theirs, so either topology is safe.

6.1 Bot token scopes

Scope	Needed for
`channels:history`	Receiving channel messages, and reading a thread's parent message for incident context.
`channels:read`	Resolving channel metadata.
`app_mentions:read`	Receiving bot mentions.
`chat:write`	Posting the answer back into the thread.
`users:read`	Resolving user IDs to display names during the optional history back-fill (Appendix D).

For a private channel, substitute the `groups:*` equivalents.

6.2 Event subscriptions

Event	Delivered to	Handler behaviour
`message.channels`	Ingestion endpoint	Appends to the thread document; ignores messages with a subtype (edits, joins, bot messages).
`app_mention`	Query endpoint	Answers from the knowledge base; ignores every other event type.

6.3 Endpoint verification

When you save a Request URL, Slack immediately posts a `url_verification` challenge. Both handlers answer this *before* signature verification, so the endpoint verifies successfully even if the signing secret has not been configured yet. This is what makes the ordering in §8 possible.

6.4 Installation

1. Install the app(s) to the workspace and copy the **Bot User OAuth Token** (`xoxb-…`) and the **Signing Secret** from Basic Information.
2. Invite the bot into the monitored channel: `/invite @<BOT_NAME>` . Without this, posting fails with `not_in_channel` .

7. Configuration reference

7.1 Ingestion Lambda

Variable	Required	Example	Purpose
`SLACK_SIGNING_SECRET`	Yes	`<SIGNING_SECRET>`	Verifies request authenticity. If unset the handler refuses all requests.
`S3_BUCKET_NAME`	Yes	`<S3_BUCKET>`	Bucket for thread documents and cases.
`S3_PREFIX`	No	`events/`	Raw audit prefix. Keep the default unless you have a reason.
`S3_CASES_PREFIX`	No	`cases/`	Curated prefix. **Must match the KB data source URI.**
`BEDROCK_MODEL_ID`	Yes	`<INFERENCE_PROFILE_ID>`	Generation model for extraction. If unset, extraction is skipped and nothing is ever indexed.
`BEDROCK_KB_ID`	Yes	`<KB_ID>`	Knowledge base to sync after a case write.
`BEDROCK_DATA_SOURCE_ID`	Yes	`<DATA_SOURCE_ID>`	Data source to sync. If either ID is unset the sync is skipped with a warning.
`CONFIDENCE_CUTOFF`	No	`0.4`	Minimum extraction confidence to index. Tune on real data.
`SLACK_WORKSPACE_URL`	Recommended	`<SLACK_WORKSPACE_URL>`	Workspace origin only, e.g. `https://acme.slack.com` — no path. Used to build citation permalinks; without it links degrade to non-clickable app links.
`LOG_LEVEL`	No	`INFO`	Set `DEBUG` when diagnosing.

7.2 Query Lambda

Variable	Required	Example	Purpose
`SLACK_SIGNING_SECRET`	Yes	`<SIGNING_SECRET>`	From the app that owns the bot user.
`SLACK_BOT_TOKEN`	Yes	`xoxb-…`	Posts replies and reads thread context.
`BEDROCK_KB_ID`	Yes	`<KB_ID>`	Knowledge base to query.
`BEDROCK_MODEL_ARN`	Yes	`arn:aws:bedrock:…`	Answer-generation model. **Note:** this is a full *ARN*, unlike the ingestion Lambda's `BEDROCK_MODEL_ID`, which is an ID. Supplying the wrong form is an easy mistake.
`AWS_REGION_NAME`	Yes	`<AWS_REGION>`	Region of the knowledge base. Defaults to `us-east-2` if unset — set it explicitly.
`SLACK_WORKSPACE_URL`	Yes	`<SLACK_WORKSPACE_URL>`	Required for clickable citations; the key-derived permalink path is silently skipped without it.

8. Deployment procedure

The order matters: the Lambdas need the knowledge base ID, Slack needs the Lambda endpoints, and the Lambdas need Slack's signing secret. That circle is broken by the `url_verification` behaviour described in §6.3 — create the functions first, verify the endpoints, then fill in the secrets.

1. **Enable Bedrock model access** for the generation model and Titan Text Embeddings V2 in `<AWS_REGION>`. Wait for approval.
2. **Create the S3 bucket** `<S3_BUCKET>`: block public access, enable encryption, enable versioning.
3. **Create the Knowledge Base** with Titan Text Embeddings V2 and an S3 Vectors store. Add one data source with URI `s3://<S3_BUCKET>/cases/`. Record the `<KB_ID>` and `<DATA_SOURCE_ID>`.
4. **Create the two IAM roles** using the policies in §5.
5. **Build the deployment packages**: `make lambda_zips`, producing `dist/events-lambda.zip` and `dist/questions-lambda.zip`.
6. **Create both Lambda functions**, upload the corresponding zip, attach the role, set handler `lambda_function.lambda_handler`, and **set the timeout to 60 seconds**. Set all non-secret environment variables now.
7. **Expose HTTPS endpoints** for both functions (Function URL or API Gateway route) and record them.
8. **Create and configure the Slack app(s)** per §6: scopes, event subscriptions pointing at the endpoints (each will verify immediately), then install to the workspace.
9. **Set the secrets** on both Lambdas: `SLACK_SIGNING_SECRET`, and `SLACK_BOT_TOKEN` on the query Lambda.
10. **Invite the bot** to the monitored channel.
11. **Run the smoke test** in §9.

8.1 Packaging note

Each zip is *flat*: the handler module is renamed `lambda_function.py` and its helper modules sit beside it at the archive root. The handlers import their helpers with a try/except fallback, so the same source works both as an installed Python package (for tests and the batch pipeline) and as a flat Lambda archive. Consequently the Lambda handler setting is always `lambda_function.lambda_handler`, and it never changes between releases.

```
dist/events-lambda.zip      lambda_function.py (post_events)  live_extract.py
                            slack_verify.py  prompts.py  parsing.py
dist/questions-lambda.zip   lambda_function.py (questions)  slack_verify.py  prompts.py
```

Keep the repository as the source of truth. The Lambda console's inline editor remains usable for emergency tweaks, but changes made there are lost on the next upload. Edit the repository and re-run `make lambda_zips`.

9. Verification — end-to-end smoke test

This exercises every component in order. Run it in the monitored channel after deployment. CloudWatch is the source of truth at each step.

Step 1 — ingestion

Post a root message in the channel, e.g. "*Checkout service returning 500s after the 14:02 deploy.*"

Expect in the ingestion Lambda's log: `[step=3] Slack signature verified` → `[step=5] Resolved thread identity … is_root=True` → `[step=8] Thread stored in S3 … role=alert`. An object now exists at `events/<CHANNEL_ID>/<thread_ts>.json`. No case is written — correct, the thread is unresolved.

Step 2 — extraction and gating

Reply *in that thread*: "*Root cause was a missing environment variable in the new release. Resolved by rolling back the deployment.*"

Expect: `[step=8] … role=resolution` → `[step=9] Resolution signal — running live extraction` → `Structured case stored — key=cases/… confidence=0.9…` → `Starting Bedrock ingestion job` → `Bedrock sync started — ingestionJobId=…`. An object now exists under `cases/`. Open it and confirm the structured fields are populated and that nothing sensitive survived.

> **Reply in the thread, not the channel.** Root messages are always classified `alert` regardless of wording, so a top-level message containing "resolved" will not trigger extraction. This is the single most common false alarm during first deployment.

Step 3 — indexing

In the Bedrock console, open the data source's sync history. **Expect** a completed job scanning 1 document and adding 1. If it scanned 0, the data source URI does not match where the Lambda wrote — re-check `S3_CASES_PREFIX` against the configured S3 URI.

Step 4 — retrieval and answering

Mention the bot with a question phrased from symptoms, not from the fix: "*@bot have we seen checkout 500s after a deploy before?*"

Expect in the query Lambda's log: `app_mention received` → `Question extracted: …` → `Calling Bedrock KB …` → `Retrieved passage [1/N] …` → `LLM answer (citations=N …)` → `Final answer after source rebuild` → `Slack message posted successfully`. In Slack, the reply appears **inside the thread of your mention** and ends with a `Sources:` list of clickable permalinks.

> The `--- FINAL ---` log block is exactly what was posted to Slack, and the `--- ANSWER ---` block above it is the raw model output before citations were rebuilt. Comparing the two is the fastest way to diagnose any formatting question.

Step 5 — negative control

Ask about something with no precedent: "*@bot have we had problems with the billing export?*" **Expect** an explicit statement that no similar past incident was found, *not* an invented answer. This confirms grounding is working, and is worth demonstrating to stakeholders — a system that admits ignorance is what makes the positive answers trustworthy.

10. Troubleshooting

Symptom	Cause	Fix
`REPORT … Status: timeout` with no error line	Lambda timeout below the Bedrock call duration.	Raise the timeout to 60 s. The most common first-deployment failure.
`AccessDeniedException … bedrock:InvokeModel`	Role lacks permission on the inference profile *or* on the underlying foundation model.	Apply both ARNs from §5.1.
`AccessDenied … s3:ListBucket`	Missing bucket-level list permission.	Add the `ListBucket` statement; without it missing objects masquerade as permission errors.
`BEDROCK_MODEL_ID not set — skipping live extraction`	Variable absent on the ingestion Lambda.	Set it to a generation model ID.
Cases never appear, ingestion looks healthy	No reply was classified `resolution`.	Reply *in-thread* using a trigger word (§2.2).
`Case gated out of the index — is_resolved=… confidence=…`	Working as designed: the thread was judged too weak.	None. Lower `CONFIDENCE_CUTOFF` only if the validation evidence supports it.
`Bedrock sync failed … ConflictException`	An ingestion job was already running.	Benign. The next case write starts a fresh sync that picks up both.
Answer is "*Sorry, I am unable to assist…*", `citations=0`	Retrieval returned nothing.	Check the `Debug retrieve:` lines. `0 chunks` → the knowledge base is empty or unsynced. `N chunk(s) but generation used none` → model or prompt-template issue.
Bot never replies in Slack	Usually the reply is in the thread and was missed; otherwise a Slack API rejection.	Check the thread first, then look for `chat.postMessage returned ok=false` — logged at **WARNING**, so it is easy to miss when filtering for errors. `not_in_channel` → invite the bot; `missing_scope` → add `chat:write` and reinstall.
Duplicate answers to one question	Slack retried an unacknowledged event.	Confirm the deployed code contains the `x-slack-retry-num` guard; verify `Ignoring Slack retry delivery #N` appears.
Citations missing or showing `%[n]%`	Older build, or `SLACK_WORKSPACE_URL` unset.	Set the variable on the query Lambda and redeploy the current build.

11. Security and data handling

Concern	Position
Raw Slack content	Stored unmodified under `events/`, including anything sensitive pasted into the channel. Treat this prefix at the same classification as the Slack channel itself. Restrict access; never index it.
Redaction	Performed by the extraction model, which replaces credentials, tokens, keys and customer identifiers with `[REDACTED:TYPE]` placeholders and sets a `redaction_applied` flag. This is a **single** layer — see Known gaps.
Request authenticity	Every request is HMAC-verified against the Slack signing secret, with a 5-minute timestamp window to prevent replay. Requests are refused outright if the secret is unconfigured.
Secrets at rest	Tokens live in Lambda environment variables in this design. For production, move `SLACK_BOT_TOKEN` and `SLACK_SIGNING_SECRET` into AWS Secrets Manager and grant each role read access to only its own secret.
Least privilege	Separate execution roles per function, scoped to specific prefixes and the specific knowledge base. Do not merge them.
Data residency	Slack content is sent to Amazon Bedrock for extraction and answering. Confirm this is acceptable under the client's data-handling policy, and note that a cross-region inference profile may process requests in another region.
Answer grounding	The model is instructed to answer only from retrieved cases and to state plainly when no precedent exists. Every answer carries source permalinks so engineers can verify before acting.

> **Redaction is currently a single layer.** The design calls for a second, answer-time pass using Bedrock Guardrails; this is **not implemented**. Until it is, extraction quality is the only barrier between a pasted credential and the knowledge base. Treat this as a known risk to be accepted explicitly or closed before handling production secrets, and pair it with the operational rule that credentials should never be pasted into the channel in the first place.

12. Cost model

Figures are orders of magnitude for a low-traffic channel (~6 messages/day, three years of history ≈ 6,500 messages yielding a few hundred to ~2,000 cases). Verify against current regional pricing before committing.

Component	Order of magnitude	Driver
One-time history back-fill	Tens of USD, once	One generation call per thread plus embedding a few thousand chunks.
Vector store (S3 Vectors)	Pennies / month	No idle floor; pay-as-you-go. This is why S3 Vectors was chosen over OpenSearch Serverless, whose standing cost would dwarf the workload at this scale.
Ongoing inference	Under ~USD 20–30 / month	Extraction on resolved threads plus question answering. Dominated by Bedrock tokens.
Lambda, S3, CloudWatch	Effectively free	Well inside free-tier territory at this volume.

Cost scales with resolved threads and questions asked, not with channel chatter: ordinary messages cost one small S3 write each. Set a Bedrock spend alarm before the back-fill, which is the single largest expense.

13. Known gaps — designed but not built

Stated explicitly so the client is not misled about current capability.

Gap	Consequence
Answer-time Guardrails	Redaction is single-layer (see §11).
Proactive auto-posting	The assistant answers only when mentioned. The designed trigger classifier, confidence gate and shadow-mode rollout for unprompted suggestions are not built.
Structured metadata store (DynamoDB)	No filtered lookups by service, no human "verified solution" flags, and no one-flag kill switch. Cases live only in S3 and the vector index.
Feedback capture	No 👍/👎 collection, so retrieval quality cannot yet improve from usage.
Infrastructure as code	Only an import workflow for an existing Lambda exists. Bucket, knowledge base, roles and functions are provisioned by console or by the client's own IaC.
Asynchronous extraction	Extraction runs inline in the ingestion handler. Fine at ~6 messages/day; move behind a queue before high-volume channels.
Bot mentions ingested as incidents	Questions to the bot are themselves stored as threads under `events/`. The confidence gate keeps them out of the index, but a trigger classifier is the proper filter.
Citation count in the reply footer	Counts Bedrock citation objects, which can differ from the number of source links shown. Cosmetic only.

Appendix A — Placeholder substitution table

Placeholder	Substitute with
`<AWS_ACCOUNT_ID>`	The client's 12-digit AWS account ID.
`<AWS_REGION>`	Deployment region, e.g. `us-east-1`.
`<S3_BUCKET>`	Bucket name holding `events/` and `cases/`.
`<KB_ID>`	Bedrock Knowledge Base ID.
`<DATA_SOURCE_ID>`	Data source ID within that knowledge base.
`<INFERENCE_PROFILE_ID>`	Generation model inference-profile ID.
`<FOUNDATION_MODEL_ID>`	Underlying foundation-model ID the profile routes to.
`<EVENTS_FN_NAME>` / `<QUESTIONS_FN_NAME>`	Lambda function names.
`<SIGNING_SECRET>`	Slack app signing secret (store in Secrets Manager for production).
`<SLACK_WORKSPACE_URL>`	Workspace origin, e.g. `https://acme.slack.com` — no trailing path.
`<CHANNEL_ID>`	Monitored channel ID, e.g. `C0XXXXXXX`.
`<BOT_NAME>`	Bot display name engineers will mention.

Appendix B — Extraction prompt

Reproduced verbatim from `src/oncall/prompts.py`. It is the quality bottleneck of the entire system: every downstream answer is only as good as this extraction. It runs one thread per call at temperature 0 for determinism. Because raw threads are retained under `events/`, the prompt can be improved and the corpus reprocessed at any time.

```
You are an extraction engine for an on-call knowledge base. You are given one Slack thread
from a single engineering on-call channel for the "Loyalty platform". Read the entire thread
and distill it into a single structured JSON record describing the incident and its
resolution.

OUTPUT
- Respond with ONLY one valid JSON object. No preamble, no explanation, no markdown code
fences.
- Use exactly the schema below. Include every field. Use null for unknown string fields and []
for unknown arrays.

GROUNDING - do not hallucinate
- Extract only what the thread states or clearly implies. Never invent a root cause, service
name, or solution that the messages do not support.
- If the root cause is not stated, set root_cause to null. If no fix or workaround was
reached, set solution to null and solution_type to "none".

RESOLUTION JUDGMENT
- Set is_resolved to true only if the thread reaches a concrete fix or workaround, or clearly
states the issue was resolved.
- Set it to false for ongoing, speculative, abandoned, or chatter-only threads.

REDACTION - security
- Replace secrets and credentials with a placeholder of the form [REDACTED:TYPE]: API keys,
tokens, bearer/JWT values, passwords, AWS access/secret keys, private connection strings, and
customer PII (personal emails, customer IDs, customer full names). Set redaction_applied to
true if you replaced anything.
- Preserve operational detail needed to understand the incident: service names, error
messages, resource names, command names, and internal component names are NOT secrets - keep
them.

CATEGORY
- category is the single primary system/layer of the ROOT CAUSE, chosen from this controlled
vocabulary only: application_bug, infrastructure_failure, service_communication, aws,
kubernetes, docker, argocd_deployment, datadog_monitoring, other.
- Put any additional relevant labels (secondary categories, affected technologies, service
names) in tags.

CONFIDENCE (0.0-1.0)
- 0.85-1.0: clear issue, clear root cause, and a fix/workaround the thread confirms worked.
- 0.5-0.84: clear issue and a plausible solution, but root cause unclear or fix unconfirmed.
- 0.2-0.49: issue identifiable but no real resolution.
- 0.0-0.19: ambiguous, off-topic, or chatter.

SUMMARY
- summary is one or two plain sentences capturing symptom + affected component + fix, written
for similarity search against future incident descriptions.

SCHEMA
{
  "is_resolved": true,
  "summary": "string",
  "issue": "string",
  "affected_service": "string | null",
  "category": "application_bug | infrastructure_failure | service_communication | aws |
kubernetes | docker | argocd_deployment | datadog_monitoring | other",
  "tags": ["string"],
  "root_cause": "string | null",
  "troubleshooting_steps": ["string"],
  "solution": "string | null",
  "solution_type": "fix | workaround | none",
  "confidence": 0.0,
```

```
  "permalink": "string",
  "redaction_applied": false
}
```

Appendix C — Answer prompt template

Supplied to Bedrock `retrieve_and_generate`. `$query$` and `$search_results$` are substituted by Bedrock at call time and must be preserved exactly.

```
You are an on-call assistant for the Loyalty platform. An engineer is asking for help with a
current production issue. You are given past incidents retrieved from the team's resolved-
incident knowledge base; each includes a Slack permalink to the original thread.

RULES
- Use ONLY the retrieved past incidents below. Never invent fixes, root causes, services, or
facts that are not present in them.
- If none of the retrieved incidents is a real match for the engineer's problem, say so
plainly: "I couldn't find similar past incidents in our history. Please check the runbooks or
escalate." Do not force a weak match into an answer.
- Be concise and practical. Lead with the most likely cause and fix. Use bullet points for
resolution steps, maximum 5.
- Prefer a confirmed fix over an unconfirmed workaround, and flag low-confidence or old
incidents as such.
- End with a "Sources:" line listing the permalink of every incident you drew on, so the
engineer can open the original thread and verify.
- Frame everything as leads to verify, not guarantees. The engineer decides.

Current incident and question:
$query$

Retrieved past incidents:
$search_results$
```

Appendix D — Optional: back-filling channel history

The live path only learns from threads resolved after deployment. To make the assistant useful on day one, back-fill the channel's existing history using the batch pipeline shipped with the repository.

1. `make export CHANNEL=<CHANNEL_ID>` — read-only Slack export with rate-limit backoff.
2. `make pipeline` — normalises threads, extracts a **30-thread sample**, and produces an HTML validation report.
3. **Review the report before continuing.** Confirm high-confidence rows contain real fixes, check whether useful cases sit just below the cutoff, and spot-check redaction. Fix quality in the prompt and re-run; never hand-edit extracted data.
4. `make extract LIMIT=0 && make validate` — full extraction.
5. Publish the indexable cases to `s3://<S3_BUCKET>/cases/` using the same key layout as the live path (`cases/{channel_id}/{thread_ts}.json`), then sync the data source. The repository's uploader writes exactly this layout, so a thread later re-resolved in Slack updates its case rather than duplicating it.
6. `make holdout` — hides recent resolved incidents, queries the retriever with symptom text only, and reports a hit-rate. Use this as the go/no-go measure before rolling the assistant out to the on-call team.

> Clear any test cases out of `cases/` and re-sync before back-filling, so smoke-test artefacts do not pollute retrieval.

Appendix E — Items to confirm before delivery

Item	Why it is open
Slack event delivery topology (§6)	The repository does not record whether the reference deployment uses two Slack apps or one app with a dispatcher. Confirm, then keep only the applicable option.
HTTPS entry point	Evidence exists for both Lambda Function URLs and an API Gateway HTTP API stage. Confirm which the client should standardise on.
Lambda memory sizing	256 MB is recommended based on observed usage under 100 MB; confirm against the reference deployment's configuration.
Region	S3 Vectors availability should be re-verified in the client's target region at deployment time.

Document version 1.0 — generated from the deployed implementation. Regenerate when the handlers or prompts change.